仿采精灵 Python SDK 文档

仿采精灵是琏雾平台的系列产品，兼具多传感设备集成仿真与数据采集功能。它能在无传感硬件时通过仿真设备支持物联网应用软件开发调试，且能无缝切换接入不同协议的真实硬件，实现高并发、高精度实时数据采集与集中管理，为后续分析提供连续、可信的数据源。

1. 项目概述

FangCai Python SDK 是专为仿采精灵设计的 Python 客户端开发包。SDK 封装了与仿采精灵交互的底层通信逻辑，使开发者能够轻松获取仿采精灵配置的传感设备以及传感数据。下面对 SDK 运行所需的环境要求、项目准备工作、模块功能、综合案例分别进行介绍。

2. 环境要求

环境要求是 Python SDK 项目开发所必须的，包括开发环境和软件包。

2.1 开发环境

项目开发环境如表 2-1 所示，建议版本匹配。

表 2-1 项目开发环境

环境	版本
Python	3.9 +
IDE	PyCharm 2024.1 x64
Pip	22.0.4

2.2 软件包

python 软件包详细信息如表 2-2 所示，建议版本相同。

表 2-2 软件包详细信息

软件包	版本	功能
requests	> 2.32.3	构建 http 请求
paho-mqtt	> 2.1.0	实现 MQTT 协议通信

3. 项目准备

项目准备介绍了使用项目前需要了解或进行的准备工作，包括 SDK 项目的目录结构、仿采精灵的连接方式、项目的配置文档、SDK 的工作流程以及 SDK 的编译与运行方式。

（1）项目结构

将 LianWuPy.zip 解压后，可以看到 SDK 项目的目录结构包括如下内容，**注意：打开项目时要以“LianWuPy”为根目录**。

```
LianWuPy
    config.py              配置文件
    device_api_client.py   http 封装
    main.py                主函数
    mqtt_client.py         mqtt 封装
```

（2）连接仿采精灵

单用户版仿采精灵：通过交换机连接笔记本电脑和仿采精灵；

多用户版仿采精灵：将笔记本电脑连接仿采精灵所接入的无线路由器，即可建立笔记本电脑和仿采精灵的网络连接。

（3）配置文档

在项目启动前先前往 config.py 文件填写相关配置案例如下，将 IP 地址修改为仿采精灵实际 IP 地址，如 192.168.0.2：

```
class config:
    mqtt_host: str = "192.168.0.2"
    mqtt_port: int = 1883
    mqtt_username: str = "appback"
    mqtt_password: str = "ouccs"
    http_host: str = '192.168.0.2'
    http_port: int = 8888
    description: str = '配置文件示例'
```

（4）工作流程

SDK 项目工作流程首先初始化 MQTT 客户端并建立连接，随后订阅相关的数据采集主题；接着通过 HTTP 接口调用 get_filters 获取设备类型和型号信息，调用 get_all 查询所有传感器信息，调用 get_sensor_by_instrument 获取具体传感器配置；然后根据传感器参数构建采集命令，通过 MQTT 协议发送控制指令；最后接收传感器上传的数据并进行解析处理，完成整个数据采集周期。调用流程

如图 3-1 所示。

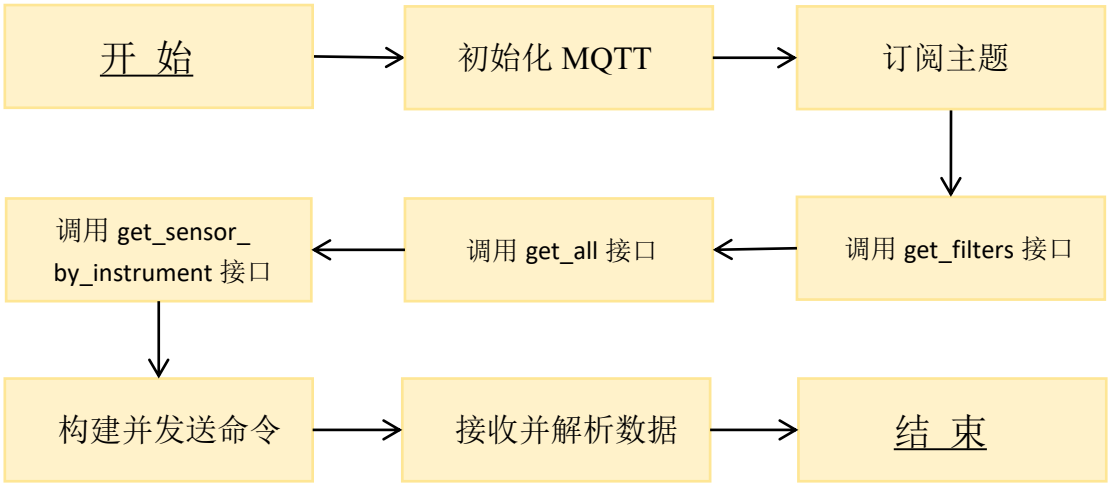


图 3-1 工作流程

4.模块功能详解

模块功能详解部分对 SDK 项目里主要模块进行介绍，模块包括 HTTP 模块和 MQTT 模块。

4.1 HTTP 模块

HTTP 模块采用 requests 实现，相关核心方法的定义在“device_api_client.py”，相关核心方法见表 4-1，包括 3 个核心方法，分别在 4.1.2、4.1.3、4.1.4 小节详细介绍。

表 4-1 HTTP 模块核心方法

方法签名	描述	对应 API 路径
get_filters()	获取设备类型、型号及网关信息的过滤器字典	/starLightOS/device_management/getfilters
get_all()	分页查询设备列表，支持多条件过滤（网关号、类型等）	/starLightOS/device_management/getall
get_sensor_by_instrument()	根据设备 ID 批量获取传感器详细定义	/starLightOS/device_management/getsensorbyinstrument

4.1.2 get_filters()

（1）函数声明: def get_filters(self) -> List[Dict[str, Any]]

（2）函数作用：查询设备类型及设备型号，调用接口后可以查询所有网关信息，目前可支持的所有设备型号，设备类型，以及设备生产商。

（3）接收参数说明: 无

（4）返回结果说明:如表 4-2 所示。

表 4-2 get_filters 返回结果

类	字段	数据类型	描述
gates	gateQos	int	网关 QoS 等级
	gateType	int	网关类型
	gateNo	int	网关编号
	gateName	str	网关名称
instrumentTypes	instrumentType	str	仪器类型
	model	str	设备型号
manufacturers	manufacturer	str	制造商名称

（5）使用示例:

```
response:List[Dict[str, Any]] = get_filters()
```

4.1.3 get_all()

（1）函数声明:

```
def get_all(self, page: int = 1, limit: int = 10,
            user_id: int = 1, gateway_no: int = 1,
            instrumentType: str = '', model: str = '',
            interfaceType: str = '', sortType: str = '') -> Dict[str, Any]
```

（2）函数作用：查询传感设备信息，该接口支持条件查询。

（3）接收参数说明:如表 4-3 所示。

表 4-3 get_all 参数字段信息

参数名	参数类型	描述
limit	int	必填，每页条数
page	int	必填，页号
userId	int	必填，固定为 1
name	str	非必填，条件查询：根据设备名称
instrumentType	str	非必填，条件查询：根据设备类型
model	str	非必填，条件查询：根据设备型号

interfaceType	str	非必填，条件查询：根据接口类型
gatewayNo	str	非必填，条件查询：根据网关号
sortType	str	非必填，条件查询：根据网关号

（4）返回结果说明:如表 4-4 所示。

表 4-4 get_all 返回结果信息

字段	数据类型	描述
id	int	传感器数据库主键 ID
instrumentId	int	设备数据库主键 ID
instrumetRelId	int	设备相对 ID
name	str	传感器名称
instrumentType	str	传感器类型
interfaceType	str	接口类型
changeable	str	可否改变
model	str	设备型号
sensornum	int	传感器数量
gateNo	int	网关编号
gateName	str	网关名称

（5）使用示例：

```
response: Dict[str,Any] = get_all(1,10,1,1)
```

4.1.4 get_sensor_by_instinstrument

（1）函数声明:

def get_sensor_by_instrument(self, device_ids: List[int], page: int = 1, limit: int = 10) -> Dict[str, Any]

（2）函数作用：根据设备的数据库主键 ID 查询传感器信息址和端口。

（3）接收参数说明:get_sensor_by_instinstrument 如表 4-5 所示。

表 4-5 get_sensor_by_instinstrument 参数字段信息

参数名	参数类型	描述
limit	int	必填，用于分页

page	int	必填，用于分页
str	str	必填，一个 json 字符串，包含查询实际 ID 的传感器信息（注意此处的 ID 为设备的数据库主键 ID 并非 instrumentId），原始数据会经过 param 编码

（5）返回结果说明:如表 4-6 所示。

表 4-6 get_sensor_by_instinstrumen 字段信息

字段	数据类型	描述
id	int	传感器数据库主键 ID
globalId	int	全局 ID
relativeId	int	相对 ID
instrumentId	int	设备数据库主键 ID
name	str	名称
unit	str	单位
decimalPlaces	int	小数位数
字段	数据类型	描述
simulationType	str	模拟类型
isVirtual	int	是否虚拟

（5）使用示例:

```
response:Dict[str,Any] = get_sensor_by_instrument([1,2,3],1,10)
```

4.2 MQTT 模块

MQTT 模块采用 paho-mqtt 实现，相关方法的实现在“mqtt_client.py”，相关方法如表 4-7 所示。

表 4-7 MQTT 相关方法信息

回调方法名	描述
def on_connect(self, client, userdata, flags, rc)	MQTT 连接成功回调
def on_message(self, client, userdata, msg)	接收 MQTT 消息回调

4.2.1 on_connect()

（1）函数声明：

def on_connect(self, client, userdata, flags, rc)

（2）函数作用：MQTT 连接成功回调。

（3）回调数据类型：如表 4-8 所示。

表 4-8 on_connect 回调数据字段信息

字段	数据类型	描述
client	Client	回调的客户端实例
userdata	Any	Client()或 user_data_set() 中设置的私有用户数据
flags	dict	broker 发送的响应标志
rc	int	连接结果

（4）使用示例：

```
self.client.on_connect = self.on_connect
```

4.2.2 on_message

（1）函数声明：

def on_message(self, client, userdata, msg)

（2）函数作用：接收 MQTT 消息回调。

（3）回调数据类型：如表 4-9 所示。

表 4-9 on_message 回调数据字段信息

字段	数据类型	描述
client	Client	回调的客户端实例
userdata	Any	Client()或 user_data_set() 中设置的私有用户数据
msg	MQTTMessage	收到的信息，涵盖 members topic、payload、qos、retain

（4）使用示例：

```
self.client.on_message = self.on_message
```

5.综合案例

5.1 配置文件获取数据案例

说明:该案例的入口文件在“LianWuPy/main_static.py”中。

该案例通过配置文件获取传感器信息并获取传感器数据，在仿采精灵传感设备列表中找到对应的传感器信息，例如，温湿度传感设备的传感器信息如图 5-1 所示，包括 1 个温度传感器，1 个湿度传感器，传感器信息包括传感器全局 ID、传感器名称、数据单位。



图 5-1 传感器列表

修改 LianWuPy/sensors_config.json，如图 5-2 所示，和传感器列表对应，其中 globalId 为传感器全局 ID，name 为传感器名称，unit 为数据单位，deviceName 为传感设备名称。

```
[
  {
    "globalId": 1,
    "name": "温度",
    "unit": "℃",
    "deviceName": "温湿度1"
  },{
    "globalId": 2,
    "name": "湿度",
    "unit": "%RH",
    "deviceName": "温湿度1"
  }
]
```

图 5-2 传感器配置文件

项目编译运行后效果如图 5-3 所示，采集周期设置为 1 秒，程序每隔 1 秒接收到仿采精灵传回的温湿度传感器数据。

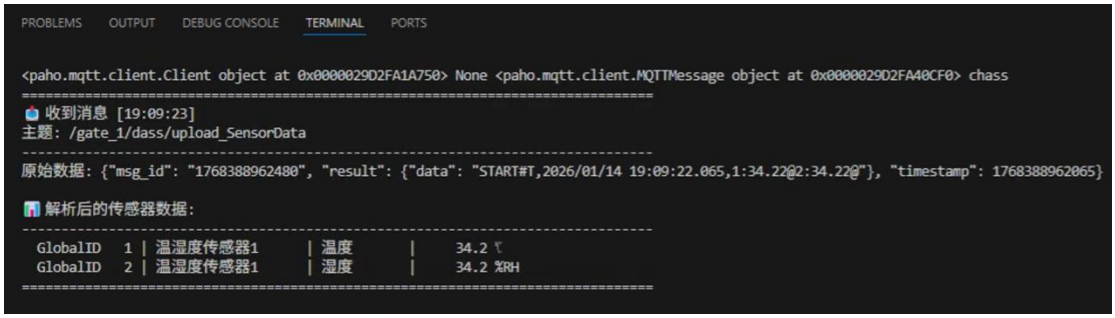


图 5-3 程序接收配置传感器数据

5.2 网络接口获取数据案例

说明:该案例的入口文件在 main.py 中。

该案例通过网络接口获取全部传感器信息并获取传感器数据，项目编译运行后效果如图 5-4 所示，采集周期设置为 1 秒，程序每隔 1 秒接收到仿采精灵传回的温湿度传感器数据。

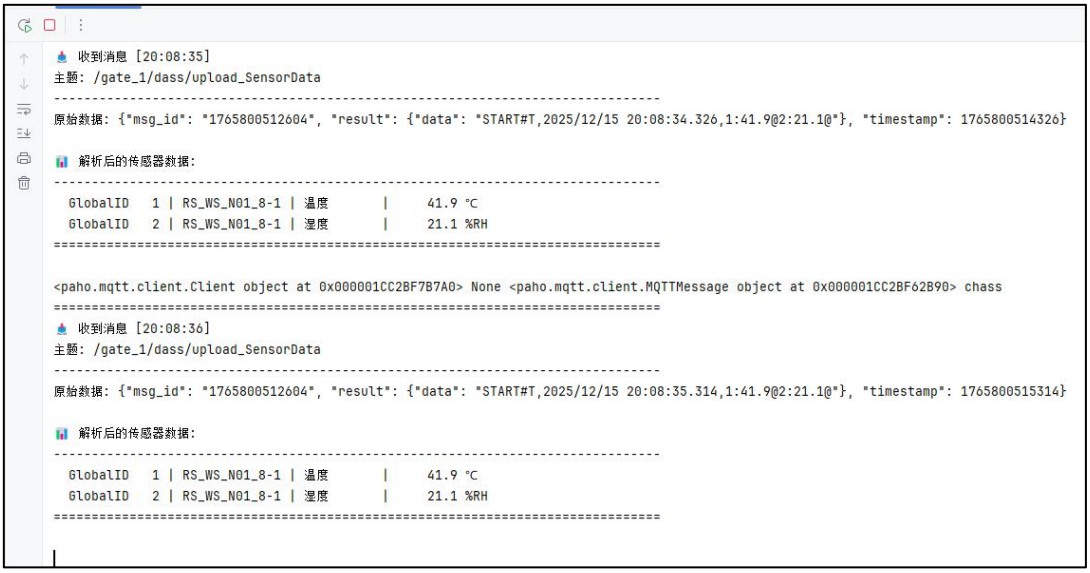


图 5-4 程序接收全部传感器数据

```
mqtt_client = MQTTClient(mqtt_host=config.mqtt_host, mqtt_port=config.mqtt_port,
username=config.mqtt_username,password=config.mqtt_password, api_host='http://' +
config.http_host + ':' + str(config.http_port))
    gateNo = None
    filters = mqtt_client.api.get_filters()
    if filters['gates'][0]['gateName'] == '默认普通采集网关':
        gateNo = filters['gates'][0]['gateNo']
    response = mqtt_client.api.get_all(gateway_no=gateNo)
```

```
    if not response.get("success"):
        print("✗ 获取设备列表失败")
        return False
    devices = response.get("data", [])
    if not devices:
        print("✗ 没有设备")
        return False
        # 获取网关号
    mqtt_client.gate_no = devices[0].get("gateNo")
    print(f"✓ 设备数量: {len(devices)}")
    print(f"✓ 网关号: {mqtt_client.gate_no}")
    device_ids = [d.get("id") for d in devices]
    device_name_map = {d.get("id"): d.get("name") for d in devices}
    sensor_response = mqtt_client.api.get_sensor_by_instrument(device_ids)
    if not sensor_response.get("success"):
        print("✗ 获取传感器信息失败")
        return False
    sensors = sensor_response.get("data", [])
    for sensor in sensors:
        global_id = sensor.get("globalId")
        if global_id:
            instrument_id = sensor.get("instrumentId")
            sensor['deviceName'] = device_name_map.get(instrument_id, "Unknown")
            mqtt_client.sensor_map[global_id] = sensor
    print(f"✓ 传感器数量: {len(mqtt_client.sensor_map)}")
    for i, (gid, info) in enumerate(list(mqtt_client.sensor_map.items())):
        print(f"   GlobalID {gid}: {info.get('deviceName')} - {info.get('name')}
({info.get('unit')})")
    # 标记传感器信息已加载
```

```
    mqtt_client.sensor_loaded = True
    if not mqtt_client.connect_mqtt():
        return
    # 发送采集命令
    mqtt_client.send_commands()
    # 开始监听
    mqtt_client.start_listening()
```